



Cairo University
Faculty of Computers and Artificial Intelligence

CS251

Introduction to Software Engineering

Budget Management
Software Design Specifications

Version 1.0

ID	Name	Email	Mobile
20240394	Amr Tamer Abdellatif	20240394@stud.fci-cu.edu.eg	01097225047
20240093	Andro Hazem Samir	20240093@stud.fci-cu.edu.eg	01274681549
20240150	Hazem Hosny Hamed	20240150@stud.fci-cu.edu.eg	01099516007
20242450	Manal Ezzat Ahmed	20242450@stud.fci-cu.edu.eg	01100427505



CS251: Sanad

Project: Budget Management

Software Design Specification

April of 2026

Contents

Team	3
Document Purpose and Audience	3
System Models	3
I. Architecture Diagram	3
II. Class Diagram(s)	4
III. Class Descriptions	5
IV. Sequence diagrams	6
Class - Sequence Usage Table	7
V. State Diagram	8
VI. SOLID Principles	8
VII. Design Patterns	8
Tools	8
Ownership Report	8



CS251: Sanad

Project: Budget Management

Software Design Specification

Team

ID	Name	Email	Mobile
20240394	Amr Tamer Abdellatif	20240394@stud.fci-cu.edu.eg	01097225047
20240093	Andro Hazem Samir	20240093@stud.fci-cu.edu.eg	01274681549
20240150	Hazem Hosny Hamed	20240150@stud.fci-cu.edu.eg	01099516007
20242450	Manal Ezzat Ahmed	20242450@stud.fci-cu.edu.eg	01100427505

Document Purpose and Audience

I. Document Purpose

This document contains all the required system designs to build the budgeting application. The system models help specify: what systems and subsystems exist and what each component does, the interactions and blueprints of each class, the lifecycle of different instances and the different states of each object.

II. Audience

The expected audience for this are related to project management, infrastructure building and software development teams, such as:

- Project manager
- DevOps engineer
- Software developers

System Models

I. Architecture Diagram

1. System architecture design

The chosen architecture design would be feature-based/layered architecture. It breaks down the system into separate components where each component has its own layers. Due to Django's architecture design (Model-View-Template), choosing layered architecture only would lead to fat views where business logic and template rendering and routing are in one place. Choosing feature-based architecture only would not work due to the MVT structure of Django. Therefore,



CS251: Sanad

Project: Budget Management

Software Design Specification

combining Django's native architecture (layered) with modularization (feature-based) would result in a number of independent subsystems, each having separate layers for templates, views, routing and business logic.

2. System components

A. User Component

- a. Allows new customers to register.
- b. Validates data when signing in.
- c. Stores users data in the database and redirects them to the dashboard page.

B. Analytics Component

- a. Displays total balance, total income and total expense.
- b. Fetches and displays transactions and budgets data.
- c. Creates reports by analyzing income and expense data.
- d. Has a shortcut to add new transactions.
- e. Allows users to export the transactions history as a CSV file for a chosen period of time.

C. Transactions Component

- a. Displays income and expense history.
- b. Allows users to filter based on category.
- c. Allows users to add new transactions.
- d. Allows linking a transaction of type income to a savings goal.

D. Budgets Component

- a. Displays budget usage for each category.
- b. Updates after adding expenses.
- c. Sends a notification if a budget is about to run out.
- d. Allows adding new budgets.

E. Savings Goals Component

- a. Displays savings progress for each goal.
- b. Allows adding savings contribution manually.
- c. Allows adding new savings goals.
- d. Sends notification when a savings goal is reached.

F. Account Component

- a. Displays users personal information.
- b. Allows editing users information.
- c. Allows users to change account settings and check notifications.



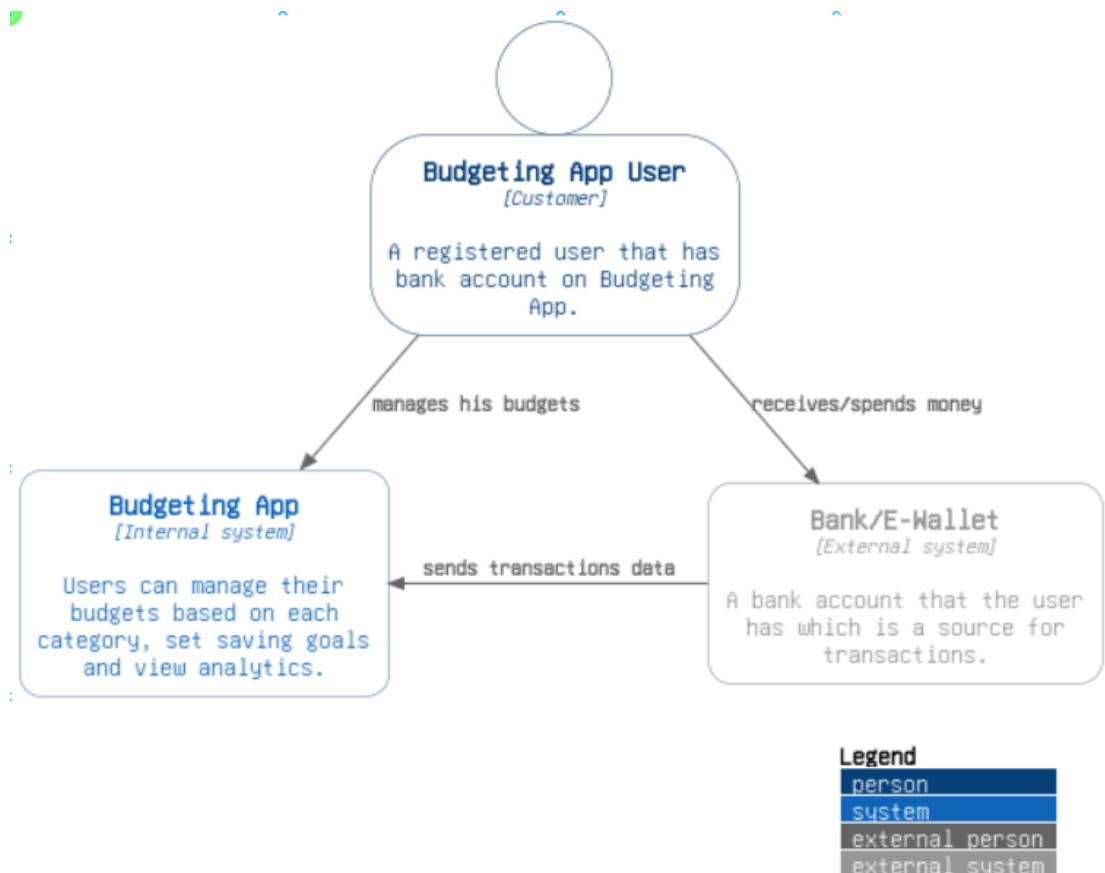
CS251: Sanad

Project: Budget Management

Software Design Specification

3. Architecture diagram (C4 notation)

A. Context layer



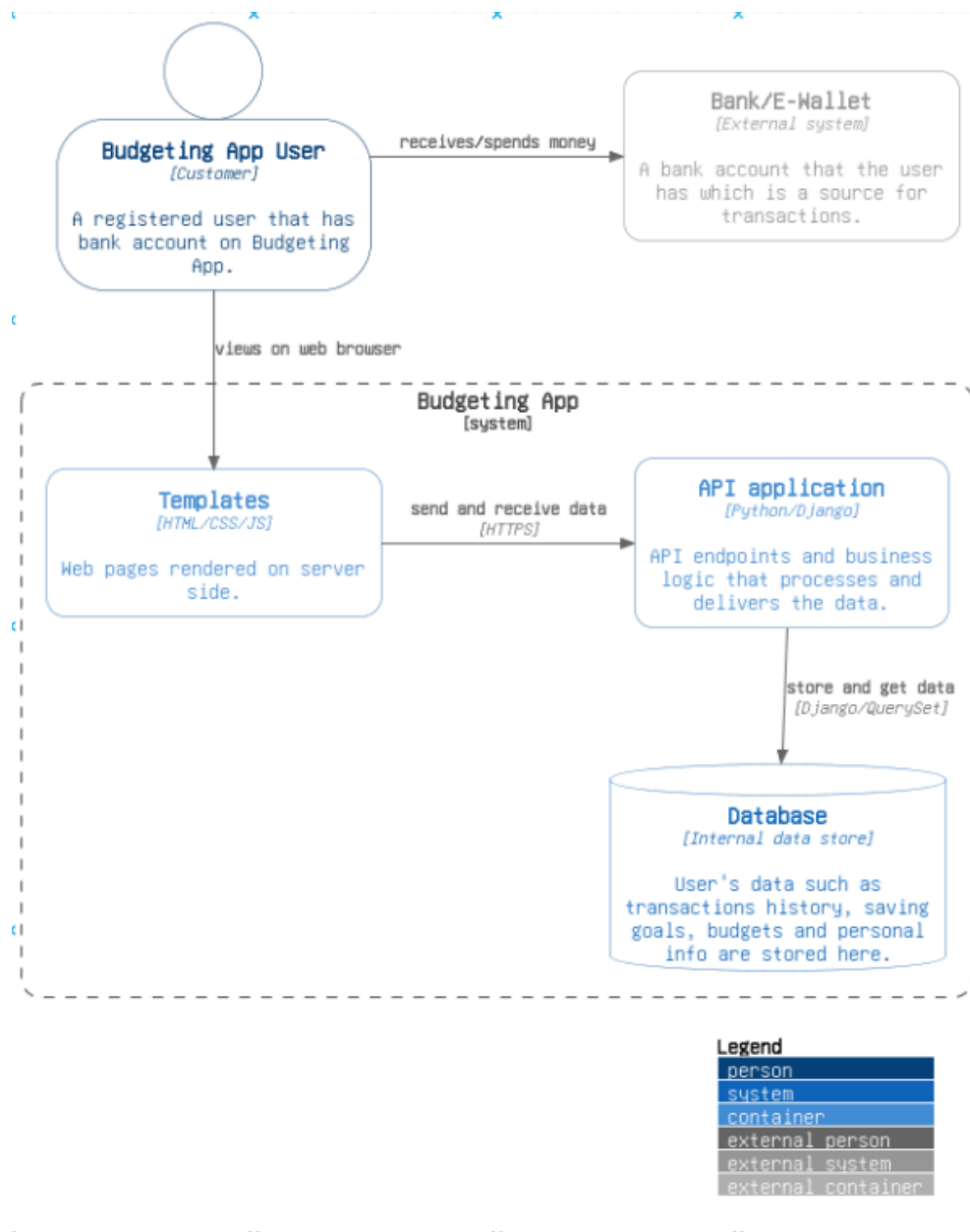


CS251: Sanad

Project: Budget Management

Software Design Specification

B. Container layer



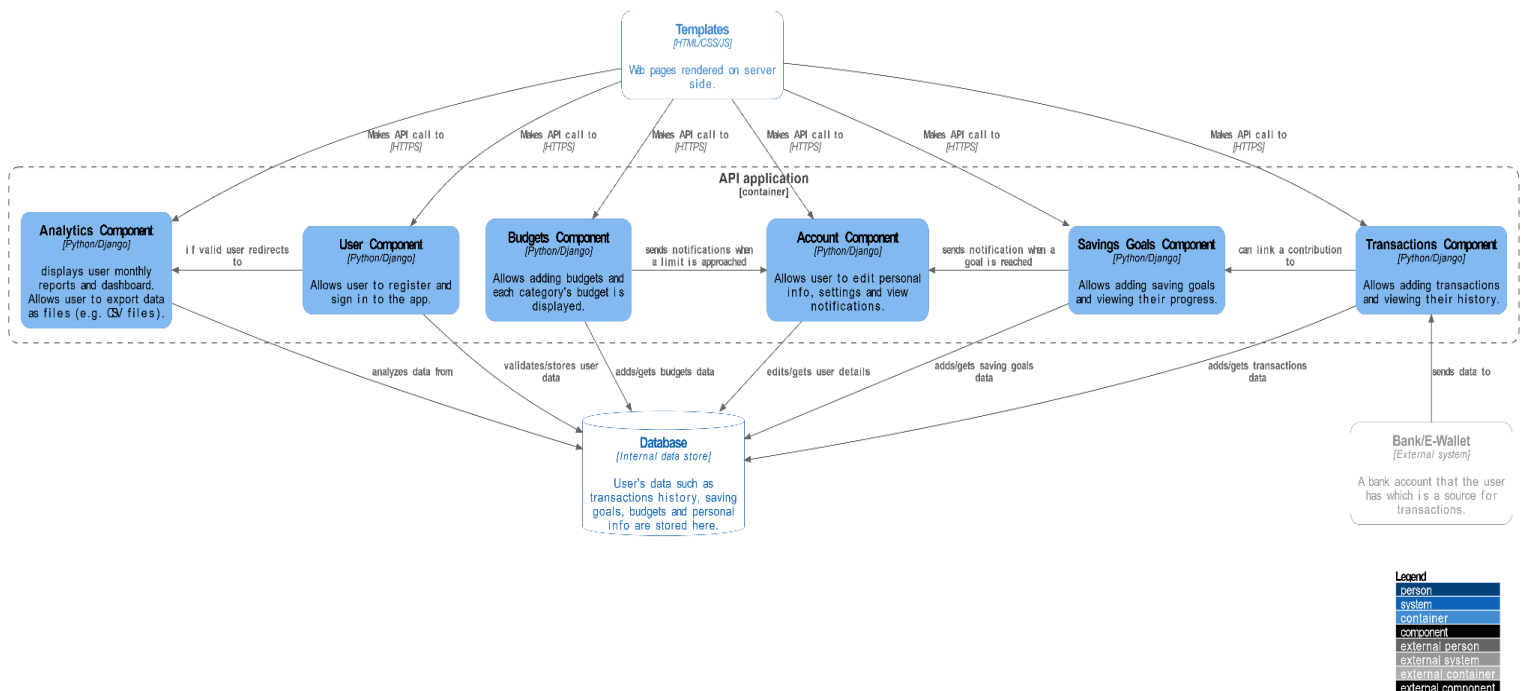


CS251: Sanad

Project: Budget Management

Software Design Specification

C. Component layer



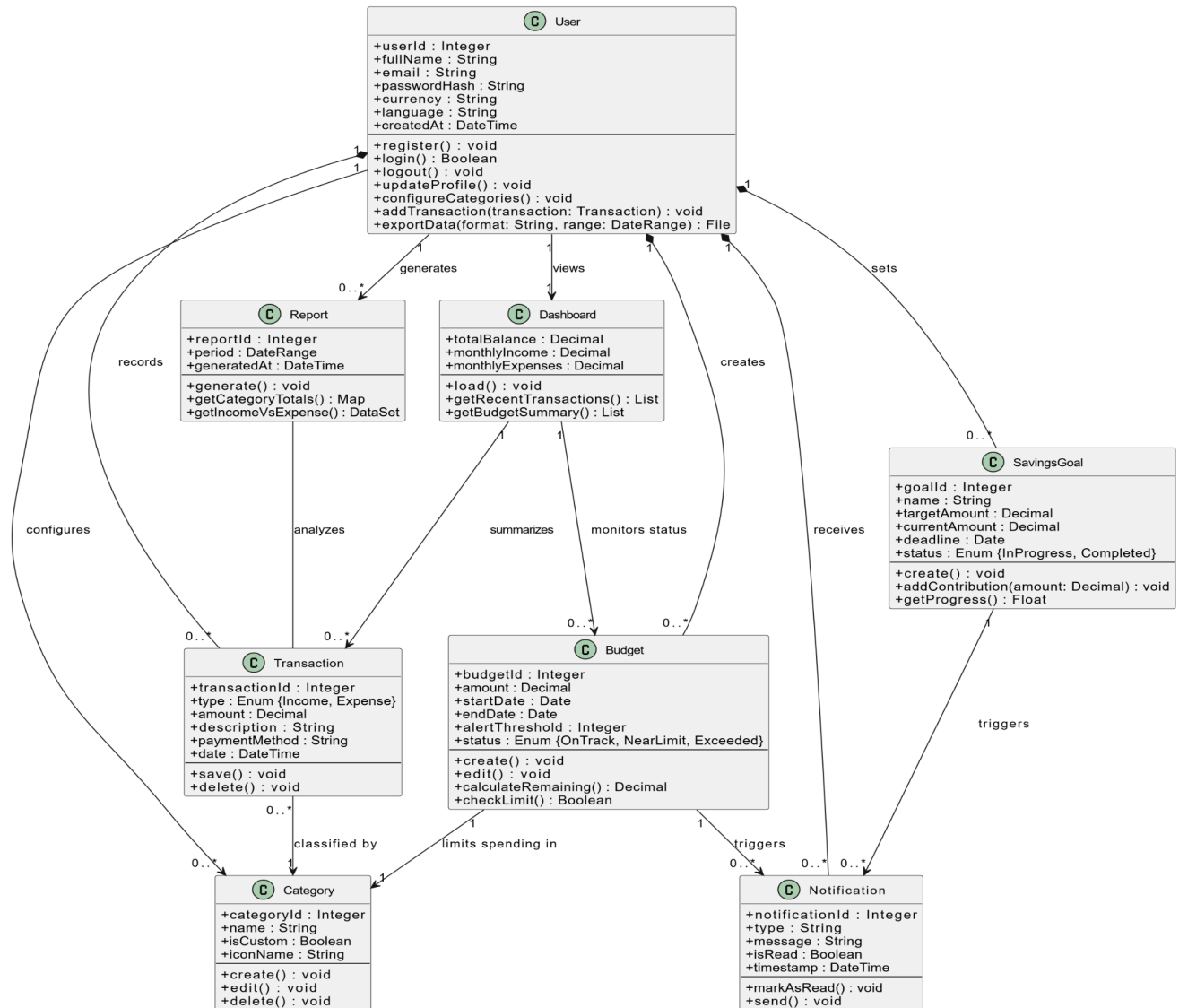


CS251: Sanad

Project: Budget Management

Software Design Specification

II. Class Diagram(s)





CS251: Sanad

Project: Budget Management

Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1.	User	The individual interacting with the system. It manages personal settings and acts as the owner of all financial data.
2.	Transaction	the details of any financial movement
3.	Category	Provides a way to organize transactions like(food,salary,etc) for better organizing. It can be a global category defined by the Admin or a personalized category defined by the User
4.	Budget	Sets financial constraints for specific categories over a set period to control spending.
5.	SavingsGoal	Tracks progress toward a specific financial target
6.	Report	Convert raw transaction data into visual insights and summaries.
7.	Notification	System alerts sent to the user regarding budget limits
8.	Dashboard	It's a view class that aggregates data for the user's home screen.



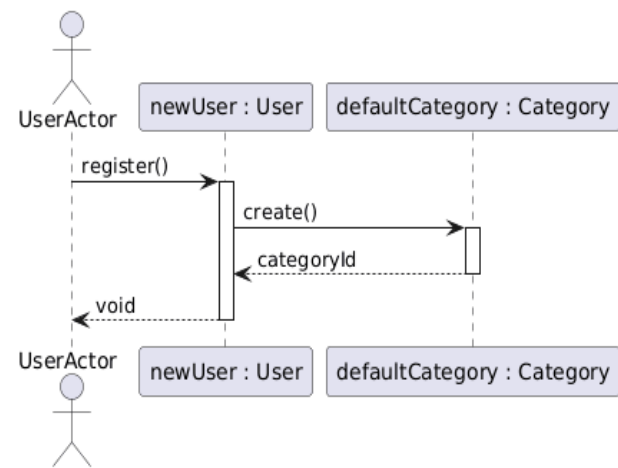
CS251: Sanad

Project: Budget Management

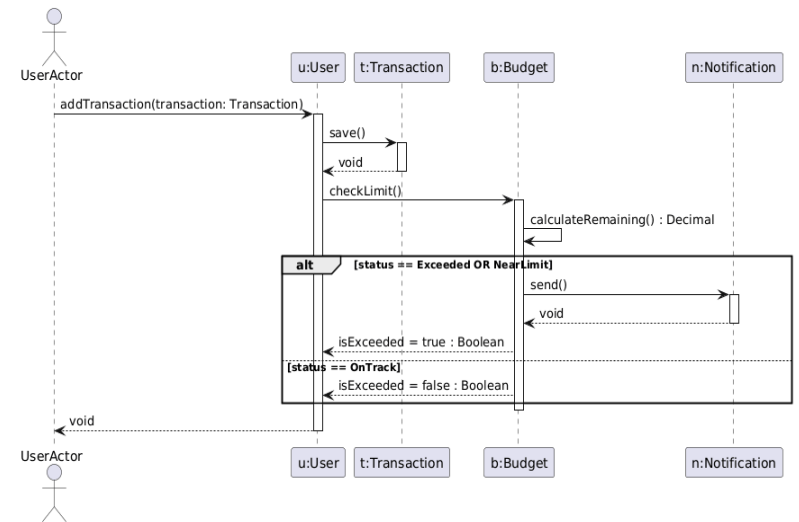
Software Design Specification

IV. Sequence diagrams

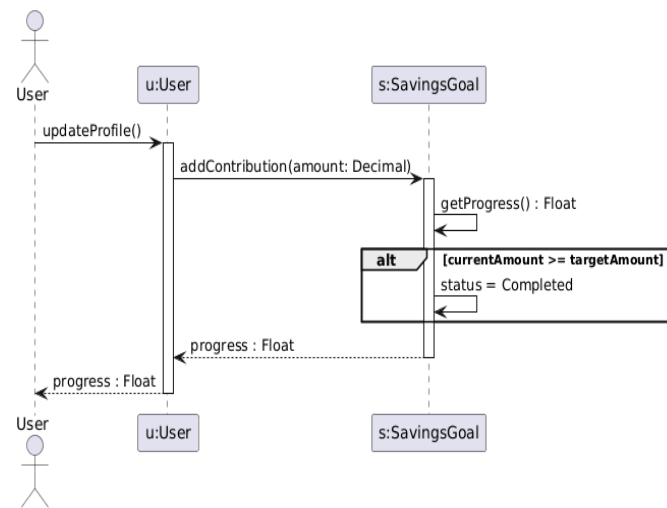
1-User Register:



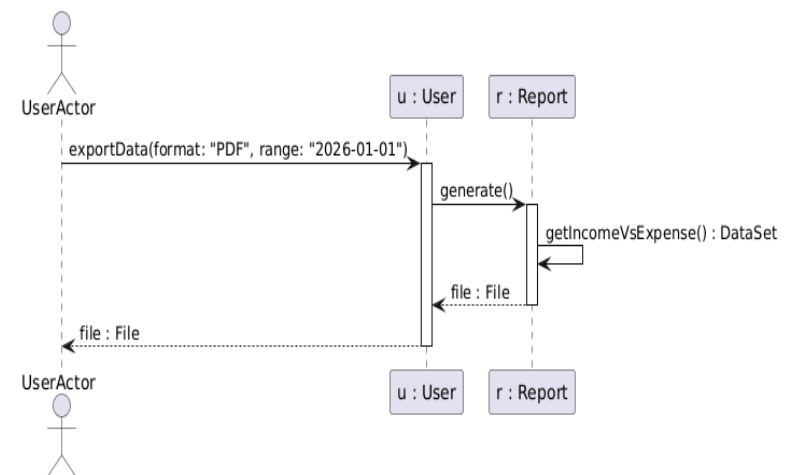
2. Adding a Transaction & Budget Check :



3-Create Savings Goal:



4-Generate Report:



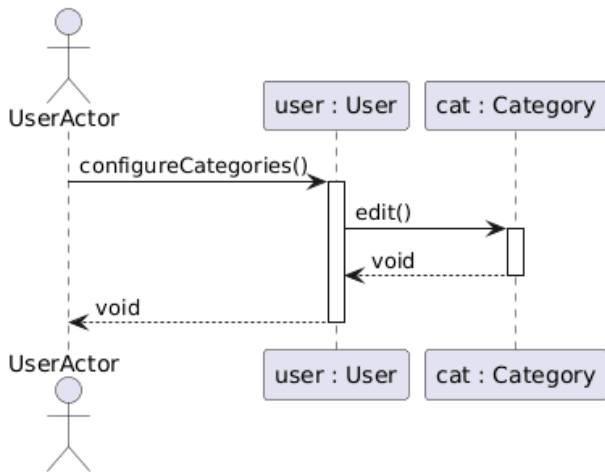


CS251: Sanad

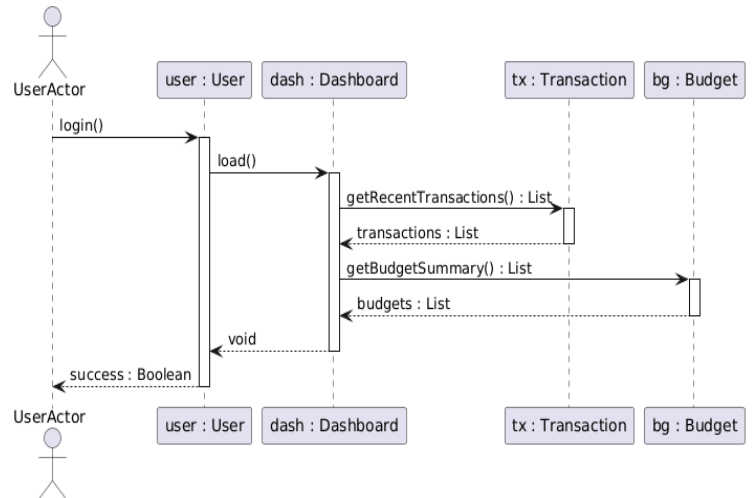
Project: Budget Management

Software Design Specification

5-Managing Categories:



6. Dashboard :



Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
1-User Register	Class user Class category	Register creat
2. Adding a Transaction	Class user Class transaction Class budget Class notification	Add transaction , save, checklimit calculateRemaining , send
3-Create Savings Goal	Class Savings Goal Class user	updateProfile AddContribute getProgress
4-Generate Report	Class user Class report	exportData Generate getIncomeVsExpense
5-Managing Categories	Class user Class category	configureCategories,edit
6. Dashboard	Class user Class transaction Class budget Class Dashboard	Login , getRecentlyTransaction Load, getbudgetSummary

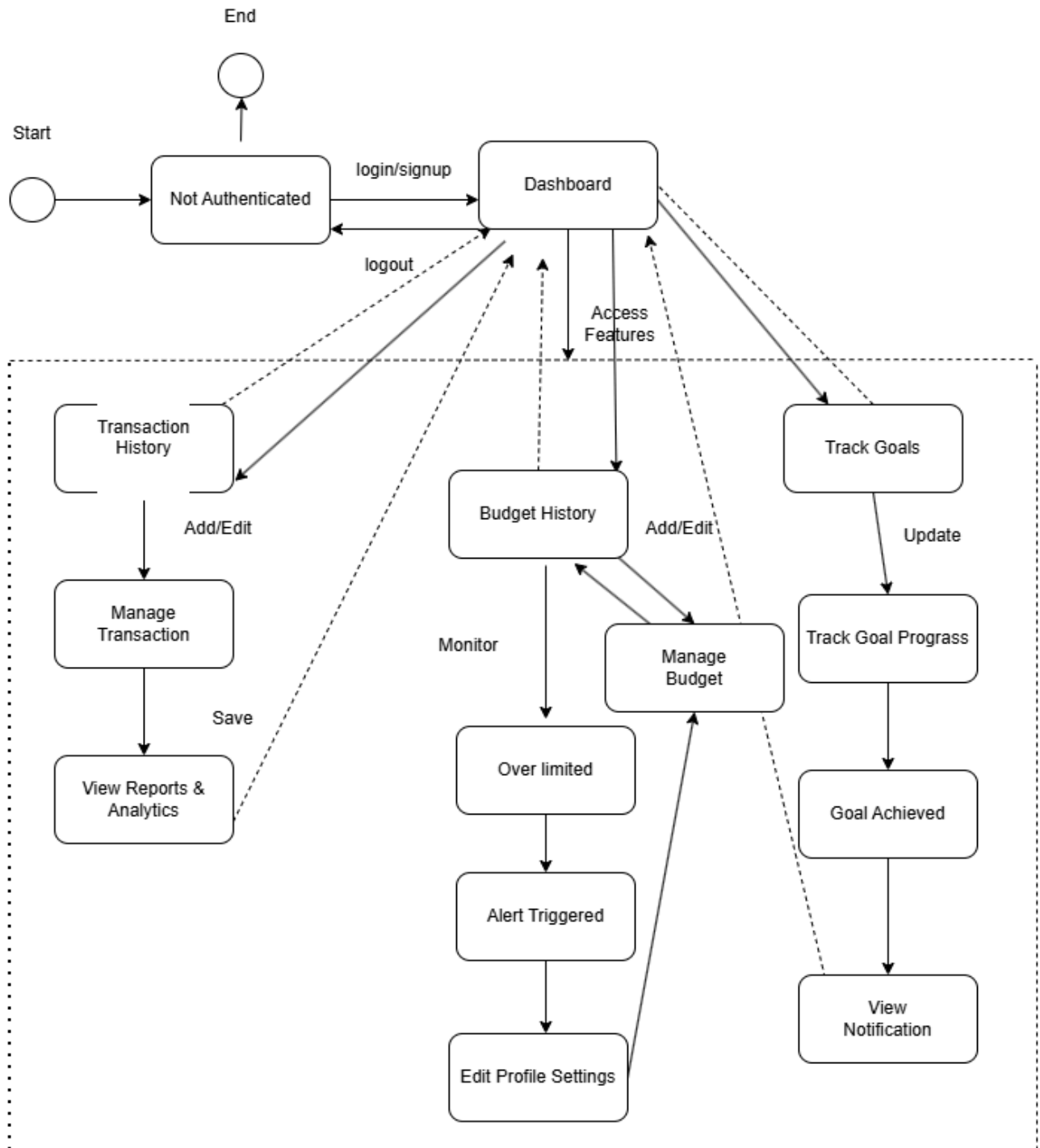


CS251: Sanad

Project: Budget Management

Software Design Specification

V. State Diagram





CS251: Sanad

Project: Budget Management

Software Design Specification

VI. SOLID Principles

1) SRP

The system was divided into smaller subsystems, each doing a single job or a group of highly related functions. An example of this is User class where it handles users data and actions only and not interfering with other functions like handling transactions and savings goals, only interacting with them.

2) LSP

Because the relationship between classes consists of composition and not inheritance, we are at no risk to break this principle.

3) DIP

Dependency inversion is packaging a group of related, frequently used functions and using them in other classes. This was achieved partially by using a transaction object in the User class to call the addTransaction() method.

VII. Design Patterns

1.Observer Pattern

Where it is used: In the Budget Tracking and Notification system.

Benefit: It ensures real-time responsiveness.

How it works: The BudgetManager acts as the Subject. The NotificationService and the DashboardUI act as Observers

2.Strategy Pattern

Where it is used: In the User Registration and Login module

Benefit: This keeps your login logic clean

How it works: You define an AuthStrategy. One strategy handles PasswordAuth, another handles OAuth, and a third handles MFA_OTP_Auth. .



CS251: Sanad

Project: Budget Management

Software Design Specification

3. Service Pattern

Where it is used: the core architectural bridge between your UI and the Database.

Benefit: It fulfills Maintainability. By separating how data is stored (Repository) from how it is processed (Service) .

How it works:

Repository: Handles the actual SQL/storage for Income, Expenses, and Goals.

Service: Handles the business logic, such as calculating Goal Completion Percentage or Remaining Budget.

4. Singleton Pattern

Where it is used: In the Database Connection Manager and User Session Manager.

Benefit: It ensures data integrity and resource efficiency.

How it works: It ensures only one instance of the database coordinator exists throughout the application's lifecycle.

Tools

- Drawio
- PlantUML

Ownership Report

Item	Owners
Amr Tamer Abdellatif	state diagram & design patterns
Andro Hazem Samir	class diagram & class descriptions
Hazem Hosny Hamed	architecture diagram & SOLID principles
Manal Ezzat Ahmed	sequence diagram &-class sequence usage table